Week 44: Convolutional Neural Networks

What they are and why, the mathematics of convolution, filters at work, pooling, and complete PyTorch and TensorFlow programs on MNIST

With pauses for questions

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for week 44

Monday lecture (2×45 min):

1. **What a convolutional network is, and why** (15 min): the same neurons, weights and biases as before – plus one assumption, that the input is an image.
2. **The mathematics of convolution** (35 min): the definition, polynomial multiplication as the first example, Toeplitz matrices, padding and stride, cross-correlation (what the libraries compute), two dimensions, a 3×3 example by hand, output sizes and parameter counts.
3. **Filters at work** (15 min): edge filters on a digit, on a drawn house and on a photograph of houses; a filter that finds windows; what a trained network learns; pooling.
4. **CNNs in code** (20 min): the same network in TensorFlow/Keras and in PyTorch on MNIST, compared with the dense network of Project 2.
5. **Fourier filters** (10 min): the convolution theorem, filtering by multiplication, the photograph in Fourier space, a CNN layer in Fourier language, and where Fourier filters are used.

Tuesday (flipped classroom): cross-validation for your Project 2 network – the week-44 exercises (deadline **Friday October 30**). Project 2 is due November 6.

Reading: Chapter 10 of the lecture notes, Sections 10.1–10.7 and 10.10–10.13 (10.7: filters at work; 10.13: Fourier filters); Goodfellow et al. Chapter 9; Raschka et al. Chapters 14–15. The code of today is in `week44.ipynb`.

Convolutional neural networks: recognising images

Convolutional neural networks (CNNs) were developed in the 1990s for character recognition. Today they are a central element of the success of deep learning, above all for images.

CNNs are very similar to the neural networks we have seen.

- They are made up of neurons with learnable weights and biases.
- Each neuron receives some inputs, performs a dot product and (optionally) applies a non-linearity.
- The whole network is a single differentiable function, from raw pixels to class scores.
- There is a loss function on the last (fully connected) layer – the softmax cross entropy for classification – and everything we know about training applies: backpropagation, gradient descent, Adam.

So what is the difference?

CNN architectures make the *explicit assumption that the inputs are images*. This lets us build certain properties into the architecture, which makes the forward pass more efficient and reduces the number of parameters enormously.

Neural networks are affine transformations – images have structure

A dense layer

A vector comes in, is multiplied by a weight matrix, a bias is added, and an activation function is applied:

$$a = f(Wx + b).$$

This works for *any* input – an image, a sound clip, a table – because whatever its shape, the input can be flattened into a vector first.

What flattening throws away

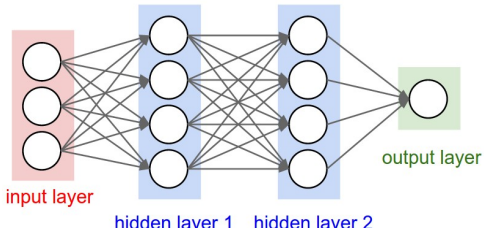
Images, sound clips and many similar data share three properties:

- they are stored as multi-dimensional arrays (the pixels);
- one or more axes have an *ordering* that matters (width and height of an image, time in a sound clip);
- one axis, the *channel* axis, holds different views of the same data (red, green and blue; left and right audio channel).

An affine transformation treats all axes the same way: the topology is not used. A **discrete convolution** is a linear transformation that *keeps* the ordering. It is *sparse* (only a few input units contribute to a given output unit) and it *reuses parameters* (the same weights are applied at many places in the input).

Regular neural networks do not scale to full images

- A colour image of 32×32 pixels has $32 \times 32 \times 3 = 3072$ numbers: every neuron of the first hidden layer needs 3072 weights. Still manageable.
- A 200×200 colour image: 120 000 weights *per neuron*. With a few hundred neurons we are at tens of millions of parameters – wasteful, and an invitation to overfit.



A regular three-layer neural network: every input connected to every hidden neuron.

A sound example

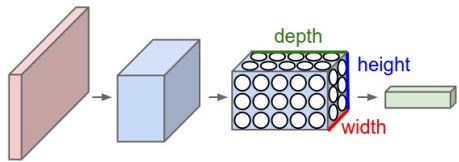
An input of length $d = 10^6$ and one hidden layer of 10^4 nodes: the weight matrix has $10^4 \times 10^6 = 10^{10}$ entries. With the 10^4 biases and the output layer,

$$\mathrm{NumberParameters} = 10^{10} + 10^4 + 10^4 + 1 \approx 10^{10},$$

ten billion parameters – to decide whether the clip is a human voice.

The CNN answer: neurons arranged in three dimensions

- The layers of a CNN have neurons arranged in *width, height and depth* – a volume. ("Depth" here is the third dimension of one layer, not the number of layers.)
- The input volume of a 32×32 colour image is $32 \times 32 \times 3$.
- A neuron is connected only to a *small region* of the layer before it – not to all of it.
- At the end the volume is reduced to a single vector of class scores, $1 \times 1 \times 10$ for ten classes.



Every layer transforms a 3D input volume into a 3D output volume. The red input layer holds the image: width and height are the image size, the depth is 3 (R, G, B).

Two principles behind the construction

Locality: neighbouring pixels are related, far-away ones much less. **Repetition**: the same pattern – an edge, a corner – appears at many places. So: connect a neuron to a small neighbourhood only, and use the *same* weights at every place. That operation is a *convolution*, a *filtering*.

Filters: hand-made in signal processing, learned in a CNN

In signal and image processing one *designs* filters: a filter is a small array of weights, and "applying the filter" is a convolution of the image with it. There are filters for smoothing, for edge detection, for reshaping frequencies – hand-crafted, from experience.

The idea of a convolutional network

Do not design the filters: *learn* them from the data, use many of them at once, and put non-linear activation functions between layers of filters so that simple patterns found by the first layer (edges) can be combined into more complex ones (corners, windows, roofs, houses) by the next.

Today we first make the word "convolution" precise (Part 2), then look at what hand-made filters do to a digit and to a house (Part 3), and finally let the libraries learn the filters (Part 4).

The layers used to build a CNN

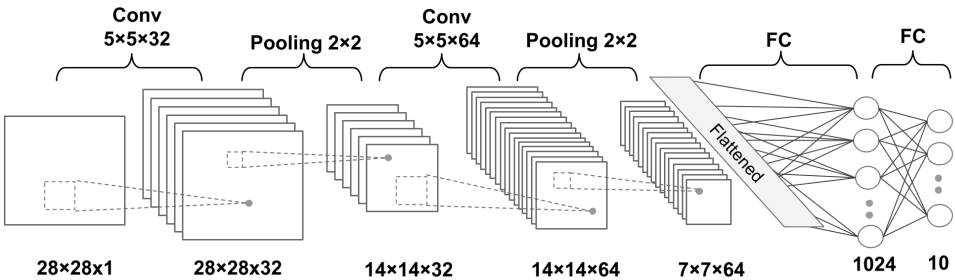
A simple CNN is a sequence of layers, and every layer transforms one volume of activations into another through a differentiable function. Three kinds of layer, stacked:

- **INPUT** $[32 \times 32 \times 3]$: the raw pixel values, width 32, height 32, three colour channels.
- **CONV**: each neuron computes a dot product between its weights and a small region of the input it is connected to. With 12 filters the output is $[32 \times 32 \times 12]$.
- **RELU**: the elementwise $\max(0, x)$; the size is unchanged, $[32 \times 32 \times 12]$.
- **POOL**: downsampling along width and height, for example to $[16 \times 16 \times 12]$.
- **FC** (fully connected): computes the class scores, $[1 \times 1 \times 10]$; every neuron sees all the numbers of the previous volume, as in an ordinary network.

Which layers have parameters

CONV and FC have weights and biases, trained by gradient descent so that the class scores agree with the labels. RELU and POOL are fixed functions with no parameters. CONV, FC and POOL have *hyperparameters* (filter size, number of filters, stride, ...); RELU has none.

A deep CNN model (from Raschka et al.)



Convolution – pooling – convolution – pooling – flatten – fully connected – fully connected. The volumes shrink in width and height and grow in depth:
$28 \times 28 \times 1 \to 28 \times 28 \times 32 \to 14 \times 14 \times 32 \to 14 \times 14 \times 64 \to 7 \times 7 \times 64 \to 1024 \to 10$. This is the network we will run in PyTorch and Keras in Part 4, with 3×3 filters instead of 5×5.

The mathematics of CNNs: convolution

Convolution is an operation on two functions that produces a third: it expresses how the shape of one is modified by the other. It appears in statistics, signal processing, differential equations, linear algebra – and machine learning.

Definition, one dimension

For a continuous input $x(a)$ and a *weight function* or *kernel* w,

$$y(t) = \int x(a)\, w(t-a)\, da, \qquad \text{written } y = x * w.$$

The discrete version, for sequences, is

$$y(t) = \sum_{a=-\infty}^{\infty} x(a)\, w(t-a).$$

- The operation is *commutative*: $x * w = w * x$ (substitute $a \to t - a$ in the sum).
- It is *linear* in each argument.
- Undoing a convolution is called deconvolution.

What does it mean, and how can we use it? Let us look at a familiar example first.

Example 1: polynomial multiplication

Multiply a polynomial of degree two by one of degree three,

$$p(t) = \alpha_0 + \alpha_1 t + \alpha_2 t^2, \qquad s(t) = \beta_0 + \beta_1 t + \beta_2 t^2 + \beta_3 t^3.$$

The product is a polynomial of degree five, $z(t) = \delta_0 + \delta_1 t + \cdots + \delta_5 t^5$, with coefficients

$$\begin{aligned}
\delta_0 &= \alpha_0\beta_0 \\
\delta_1 &= \alpha_1\beta_0 + \alpha_0\beta_1 \\
\delta_2 &= \alpha_0\beta_2 + \alpha_1\beta_1 + \alpha_2\beta_0 \\
\delta_3 &= \alpha_1\beta_2 + \alpha_2\beta_1 + \alpha_0\beta_3 \\
\delta_4 &= \alpha_2\beta_2 + \alpha_1\beta_3 \\
\delta_5 &= \alpha_2\beta_3.
\end{aligned}$$

Look at the indices: in every product the two indices *add up to the index on the left*.

Example 1 continued: the product is a discrete convolution

Set $\alpha_i = 0$ outside $i \in \{0, 1, 2\}$ and $\beta_j = 0$ outside $j \in \{0, 1, 2, 3\}$. Then every line of the previous frame is one instance of

$$\delta_j = \sum_{i=-\infty}^{\infty} \alpha_i \, \beta_{j-i} = (\alpha * \beta)_j,$$

a discrete convolution of the two coefficient sequences. Since only three α's are non-zero,

$$\delta_i = \sum_{k=0}^{m-1} \alpha_k \, \beta_{i-k}, \qquad m = 3,$$

where m is the length of $\boldsymbol{\alpha}$ and $n = 4$ is the length of $\boldsymbol{\beta}$.

Polynomial multiplication *is* convolution

Not an analogy: multiplying two polynomials convolves their coefficient sequences. The length of the result is $m + n - 1 = 6$.

Example 1 continued: a matrix–vector product

Only finitely many coefficients are non-zero, so the convolution can be written as a matrix times a vector,

$$\boldsymbol{\delta} = \begin{bmatrix} \alpha_0 & 0 & 0 & 0 \\ \alpha_1 & \alpha_0 & 0 & 0 \\ \alpha_2 & \alpha_1 & \alpha_0 & 0 \\ 0 & \alpha_2 & \alpha_1 & \alpha_0 \\ 0 & 0 & \alpha_2 & \alpha_1 \\ 0 & 0 & 0 & \alpha_2 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} \quad \text{or, equally,} \quad \boldsymbol{\delta} = \begin{bmatrix} \beta_0 & 0 & 0 \\ \beta_1 & \beta_0 & 0 \\ \beta_2 & \beta_1 & \beta_0 \\ \beta_3 & \beta_2 & \beta_1 \\ 0 & \beta_3 & \beta_2 \\ 0 & 0 & \beta_3 \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \end{bmatrix},$$

the second form because convolution is commutative.

Toeplitz matrices

A matrix whose descending diagonals are constant, $a_{ij} = a_{i-j}$, is a *Toeplitz* matrix. Both matrices above are Toeplitz (and not square). Every linear convolution is a Toeplitz matrix times a vector – and the other way round. Toeplitz matrices are closely connected with Fourier series: convolution becomes multiplication in Fourier space.

The matrices are for *reasoning*. In code we never allocate them; we compute the sum with the few non-zero terms it has.

Pause and think 1: the polynomial product by hand and by `numpy`

Question

Take $\boldsymbol{\alpha} = (1, 2, 3)$, so $p(t) = 1 + 2t + 3t^2$, and $\boldsymbol{\beta} = (4, -1, 0.5, 2)$. Compute $\boldsymbol{\delta} = \boldsymbol{\alpha} * \boldsymbol{\beta}$ from the six lines two frames back. Then check it with three lines of Python.

Pause and think 1: the polynomial product by hand and by `numpy`

Question

Take $\boldsymbol{\alpha} = (1, 2, 3)$, so $p(t) = 1 + 2t + 3t^2$, and $\boldsymbol{\beta} = (4, -1, 0.5, 2)$. Compute $\boldsymbol{\delta} = \boldsymbol{\alpha} * \boldsymbol{\beta}$ from the six lines two frames back. Then check it with three lines of Python.

Answer

$\delta_0 = 4$, $\delta_1 = 2 \cdot 4 + 1 \cdot (-1) = 7$, $\delta_2 = 1 \cdot 0.5 + 2 \cdot (-1) + 3 \cdot 4 = 10.5$, $\delta_3 = 2 \cdot 0.5 + 3 \cdot (-1) + 1 \cdot 2 = 0$, $\delta_4 = 3 \cdot 0.5 + 2 \cdot 2 = 5.5$, $\delta_5 = 3 \cdot 2 = 6$: $z(t) = 4 + 7t + 10.5t^2 + 5.5t^4 + 6t^5$.

```
alpha, beta = np.array([1., 2., 3.]), np.array([4., -1., 0.5, 2.])
```

`np.polynomial.polynomial.polymul(alpha, beta)` *gives* `[4. 7. 10.5 0. 5.5 6.]`

`np.convolve(alpha, beta)` *gives the same six numbers.*

`np.convolve` does not know about polynomials; it computes $\sum_k \alpha_k \beta_{i-k}$. The notebook also builds the 6×4 Toeplitz matrix and multiplies: the same six numbers a third time.

Generalising: a filter w and an input x

Rename $\boldsymbol{\alpha}$ (length $m = 3$) as the **filter** or **weights** w, and $\boldsymbol{\beta}$ (length $n = 4$) as the **input** x; the result $\boldsymbol{\delta}$ becomes the **output** y. We always assume $m \leq n$: the filter is small, the input is large.

$$y(i) = (x * w)(i) = \sum_{k=0}^{m-1} w(k)\, x(i-k).$$

A problem with the indices

x has the components $x(0), x(1), x(2), x(3)$, and y has six, $i = 0, \ldots, 5$. The sum then asks for $x(-1)$, $x(-2)$ (for $i = 0$) and $x(4)$, $x(5)$ (for $i = 5$), which do not exist. For the polynomials they were simply zero. In general we have to *decide* what they are.

Padding

The solution is **padding**: add P zeros before $x(0)$ and P zeros after $x(3)$. With $P = 2$ the input grows from $n = 4$ to $n + 2P = 8$:

$$x(0) = 0,\ x(1) = 0,\ x(2) = \beta_0,\ x(3) = \beta_1,\ x(4) = \beta_2,\ x(5) = \beta_3,\ x(6) = 0,\ x(7) = 0.$$

Now every index exists, and for $i = 0, \ldots, 5$

$$y(i) = \sum_{k=0}^{m-1} w(k)\, x\big(i + (m-1) - k\big).$$

Check against the polynomial product

$$\begin{aligned} y(0) &= x(2)w(0) + x(1)w(1) + x(0)w(2) = \beta_0\alpha_0 + 0 + 0 = \delta_0, \\ y(4) &= x(6)w(0) + x(5)w(1) + x(4)w(2) = 0 + \beta_3\alpha_1 + \beta_2\alpha_2 = \delta_4, \\ y(5) &= x(7)w(0) + x(6)w(1) + x(5)w(2) = 0 + 0 + \beta_3\alpha_2 = \delta_5. \end{aligned}$$

The padding P is a *hyperparameter* of the architecture, like the number of hidden nodes. A second one, the *stride* S, comes in a moment.

Flip the filter, and the sum is a dot product – cross-correlation

Look at $y(0) = x(2)w(0) + x(1)w(1) + x(0)w(2)$: the input runs *forwards*, the filter *backwards*. Define the flipped filter $\tilde{w}(0) = w(2)$, $\tilde{w}(1) = w(1)$, $\tilde{w}(2) = w(0)$. Then

$$y(0) = x(0)\tilde{w}(0) + x(1)\tilde{w}(1) + x(2)\tilde{w}(2),$$

a plain **dot product** of the filter with the *patch* $x(0{:}3)$ of the input, and $y(i)$ is the dot product with the patch $x(i{:}i+m)$: slide the filter along the input, take a dot product at every position.

Cross-correlation: what the libraries compute

Nobody flips the filter in practice. Instead of the convolution $y(i) = \sum_k w(k)\,x(i-k)$ one computes

$$y(i) = \sum_k w(k)\,x(i+k),$$

the **cross-correlation**. TensorFlow, PyTorch and our own code all implement this, and all of them *call* it convolution. The two differ only by the order of the filter weights, and since the weights are *learned*, it makes no difference. Padding and stride work the same way for both.

Two dimensions

An image is a two-dimensional array X, and a filter is a small two-dimensional array W (the *kernel*). The convolution is

$$Y(i,j) = (X * W)(i,j) = \sum_m \sum_n X(m,n)\, W(i-m, j-n),$$

or, by commutativity, with the filter indices running over the small range,

$$Y(i,j) = \sum_m \sum_n X(i-m, j-n)\, W(m,n),$$

which is the form one implements. And as in one dimension, the libraries compute the **cross-correlation**

$$\boxed{Y(i,j) = \sum_m \sum_n X(i+m, j+n)\, W(m,n)}$$

and call it convolution. Read it as: put the filter with its corner at (i,j), multiply the overlapping numbers pairwise, add them up, write the sum into $Y(i,j)$. Then move the filter.

A simple example: a 3×3 input and a 2×2 filter

$$\mathrm{X} = \begin{bmatrix} x_{00} & x_{01} & x_{02} \\ x_{10} & x_{11} & x_{12} \\ x_{20} & x_{21} & x_{22} \end{bmatrix}, \qquad \mathrm{W} = \begin{bmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \end{bmatrix}.$$

The **stride** S says how far the filter moves between two positions. With $S = 1$ it starts at x_{00} and visits every 2×2 submatrix, column by column, row by row:

$$\mathrm{Y} = \begin{bmatrix} x_{00}w_{00} + x_{01}w_{01} + x_{10}w_{10} + x_{11}w_{11} & x_{01}w_{00} + x_{02}w_{01} + x_{11}w_{10} + x_{12}w_{11} \\ x_{10}w_{00} + x_{11}w_{01} + x_{20}w_{10} + x_{21}w_{11} & x_{11}w_{00} + x_{12}w_{01} + x_{21}w_{10} + x_{22}w_{11} \end{bmatrix}.$$

With numbers

$\mathrm{X} = \left(\begin{smallmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{smallmatrix}\right)$, $\mathrm{W} = \left(\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right)$: $Y_{00} = 1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 5 \cdot 4 = 37$, and $\mathrm{Y} = \left(\begin{smallmatrix} 37 & 47 \\ 67 & 77 \end{smallmatrix}\right)$. A 3×3 input and a 2×2 filter give a 2×2 output.

For pictures of every case (padding, stride, dilation) see the repository *Arithmetic of deep learning* by Dumoulin and Visin, `github.com/vdumoulin/conv_arithmetic`.

The same example as a matrix–vector product

Flatten the input into a vector X' of length 9 and write the filter as a 4×9 matrix W':

$$X' = \begin{bmatrix} x_{00} \\ x_{01} \\ x_{02} \\ x_{10} \\ x_{11} \\ x_{12} \\ x_{20} \\ x_{21} \\ x_{22} \end{bmatrix}, \qquad W' = \begin{bmatrix} w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 & 0 & 0 & 0 \\ 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} & 0 \\ 0 & 0 & 0 & 0 & w_{00} & w_{01} & 0 & w_{10} & w_{11} \end{bmatrix}.$$

Then $W'X'$ is a vector of length 4 holding exactly the four entries of Y: the convolution with stride 1 *is* this matrix–vector product (with the numbers of the previous frame: $(37, 47, 67, 77)$).

What the matrix shows

A convolutional layer is a dense layer whose weight matrix has a very special form: most entries are zero (*locality*), and the non-zero entries are the same four numbers repeated (*parameter sharing*). A dense 4×9 layer would have 36 free parameters; this one has 4.

The convolution stage: hyperparameters, output size, parameters

The input is a *volume* of width W_1, height H_1 and depth D_1 ($D_1 = 3$ for colour, 1 for grey). The convolution stage adds, besides the weights and biases of the filters, four hyperparameters:

- K, the number of filters (common values $32, 64, 128, \ldots$);
- F, the filter's spatial extent ($F \times F$);
- S, the stride;
- P, the padding.

The output volume

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad H_2 = \frac{H_1 - F + 2P}{S} + 1,$$

Parameters to train

Each filter has $F \times F \times D_1$ weights and one bias:

$$(F \times F \times D_1) \times K + K \text{ biases.}$$

Common settings

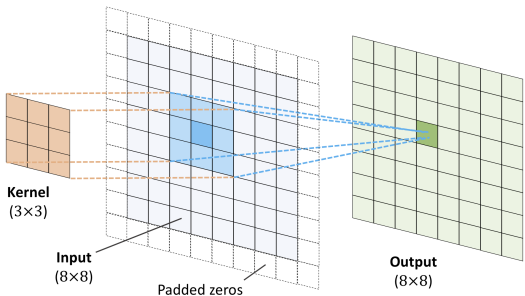
$F = 3$	$S = 1$	$P = 1$	(size unchanged)
$F = 5$	$S = 1$	$P = 2$	(size unchanged)
$F = 5$	$S = 2$	P open	(size halved)
$F = 1$	$S = 1$	$P = 0$	(mixes channels only)

A stride is a form of *subsampling*: moving the filter in hops of two is the same as moving it in hops of one and keeping every second output.

Example of a CNN setup

An input volume of $32 \times 32 \times 3$ (three colour channels). We apply a 5×5 filter ten times ($K = 10$) with stride $S = 1$ and padding $P = 0$.

- Output size: $(32 - 5)/1 + 1 = 28$, so ten feature maps of 28×28 – the output volume is $28 \times 28 \times 10$.
- Parameters per filter: $5 \times 5 \times 3 + 1 = 76$ (the last one is the bias); for ten filters, 760 parameters.
- A dense layer producing the same 7840 outputs from the 3072 inputs would have $3072 \times 7840 \approx 2.4 \times 10^7$ weights.



The general discrete convolution (Raschka et al.): a 3×3 kernel on an 8×8 input padded with one ring of zeros ($P = 1$) gives an 8×8 output.

Pause and think 2: count the parameters

Question

Same $32 \times 32 \times 3$ input. How many parameters does a layer of 32 filters of size 3×3 have ($S = 1$, $P = 0$), and what is the output volume? And for MNIST, $28 \times 28 \times 1$, with 32 filters of size 3×3, $S = 1$, $P = 1$?

Pause and think 2: count the parameters

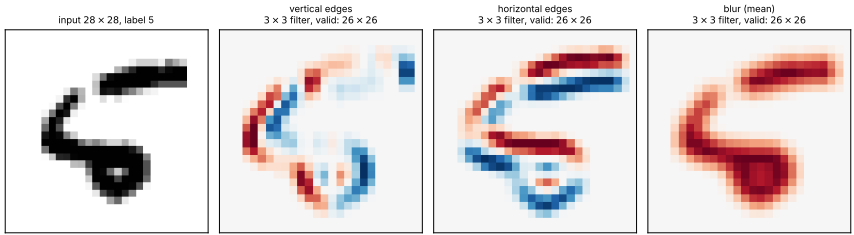
Question

Same $32 \times 32 \times 3$ input. How many parameters does a layer of 32 filters of size 3×3 have ($S = 1$, $P = 0$), and what is the output volume? And for MNIST, $28 \times 28 \times 1$, with 32 filters of size 3×3, $S = 1$, $P = 1$?

Answer

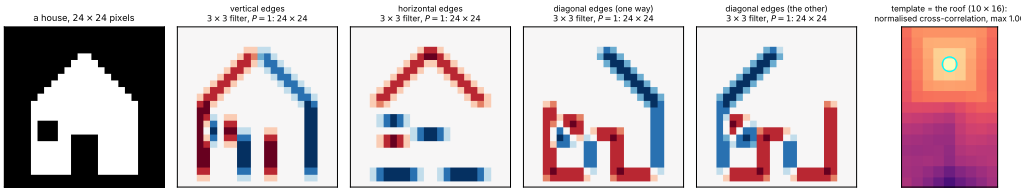
$3 \times 3 \times 3 + 1 = 28$ parameters per filter, $28 \times 32 = 896$ in total; output $(32 - 3)/1 + 1 = 30$, so $30 \times 30 \times 32$. For MNIST: $3 \times 3 \times 1 + 1 = 10$ per filter, 320 in total; with $P = 1$ the size is unchanged, $(28 - 3 + 2)/1 + 1 = 28$, output $28 \times 28 \times 32$. Note what does *not* enter the parameter count: the image size. The same 320 numbers serve a 28×28 image and a 4096×4096 one – that is parameter sharing. A dense layer from 784 inputs to 100 hidden nodes, the network of week 41, has 78 500.

Filters at work, 1: three hand-made filters on a digit



MNIST test image 8 (a five) and three 3×3 filters, no padding, so $28 \to 26$. **Vertical edges**: the Sobel filter $\left(\begin{smallmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{smallmatrix}\right)$ is a derivative along a row: positive (red) where the ink starts, negative (blue) where it ends, zero inside a stroke. **Horizontal edges**: its transpose, a derivative down a column. **Blur**: the $\frac{1}{9}$ mean of a 3×3 patch, a low-pass filter. Each output is one *feature map*. With $K = 3$ filters a layer makes all three at once, with $3 \times (9 + 1) = 30$ parameters.

Filters at work, 2: a house (`week44_house.pdf`)



A 24×24 drawing: walls, a gabled roof, a door and a window. Four 3×3 filters with $P = 1$ (the maps keep the size): the vertical-edge filter finds the walls and the sides of door and window, the horizontal one the floor, the lintels and the roof as a whole, and the two diagonal filters each pick out *one* slope of the roof. The last panel uses the roof itself, cut out of the picture, as a 10×16 filter: the normalised cross-correlation is 1.00 exactly where the roof is and smaller everywhere else – a filter that *detects a roof*.

The filters of the house, written out

Every map is the cross-correlation $Y(i,j) = \sum_{m,n} X(i+m-1,\, j+n-1)\, W(m,n)$ with $P = 1$ (one ring of zeros), so $24 \times 24 \to 24 \times 24$.

The four 3×3 edge filters

$$W_{\text{vert}} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad W_{\text{hor}} = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix},$$

$$W_{\diagup} = \begin{pmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{pmatrix}, \qquad W_{\diagdown} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{pmatrix}.$$

W_{vert} is the Sobel operator: the central difference along the row, $X(i,j+1) - X(i,j-1)$, averaged over three rows with weights $(1,2,1)$ – a smoothed $\partial X/\partial x$. $W_{\text{hor}} = W_{\text{vert}}^{T}$ is $\partial X/\partial y$. The diagonal pair are the same differences along the two diagonals (Sobel rotated by $45°$). All four have weights summing to zero: a constant region gives 0; only *changes* of intensity respond.

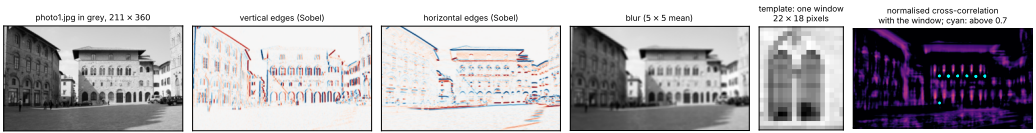
The roof as a template: normalised cross-correlation

Cut the roof out, $T = X(3{:}13,\ 4{:}20)$, a 10×16 patch, make it zero-mean with unit norm, $T \leftarrow (T - \bar{T})/\|T - \bar{T}\|$; for every patch X_{ij} of the same size do the same and take the dot product:

$$\mathrm{NCC}(i,j) = \frac{\sum_{m,n} \left(X_{ij}(m,n) - \bar{X}_{ij}\right)\, T(m,n)}{\sqrt{\sum_{m,n} \left(X_{ij}(m,n) - \bar{X}_{ij}\right)^2}} \in [-1, 1].$$

The cross-correlation of the image with T, divided by the norm of the patch: the cosine of the angle between patch and template. It is 1 exactly where the patch *is* the template up to brightness and contrast – a detector. The same formula finds the windows in the photograph (next frame). In code: `scipy.signal.correlate2d(X, W, mode="valid")` for the maps; the patch means and norms by two more correlations with a matrix of ones.

Filters at work, 3: houses in a photograph (`week44_photo.pdf`)



The photograph from the week-45 folder (`photo1.jpg`, in grey, 211×360 pixels). The Sobel filters draw the vertical lines (walls, window edges, people) and the horizontal ones (roof lines, floors, arches); the blur removes the texture of the stone. Right: one double-arched window, 22×18 pixels, cut from the photograph and used as the filter. The normalised cross-correlation lights up at *every* window of that type – six on the middle building, one on the left – and nowhere else above 0.7. A filter that finds windows is the beginning of a filter that finds houses.

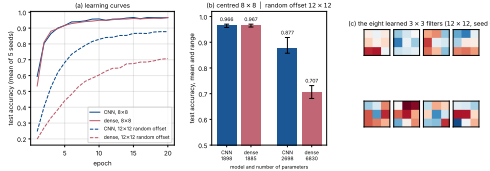
From hand-made filters to learned ones

What we did by hand

Edge filters, a blur, a roof, a window: each is a small array of weights, applied everywhere by the same formula. Choosing them took us knowledge of what a house looks like.

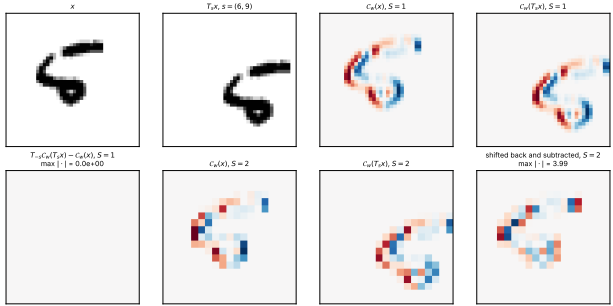
What a CNN does

It starts from *random* filters and adjusts them by gradient descent on the classification loss. Trained on digits, the first-layer filters become edge detectors – opposed-sign pairs, oriented – without anybody asking for them (right, from the notebook). The second layer combines edges into corners and strokes, the third into shapes, and the dense layers at the end decide between the classes. A network trained on houses would learn the window and the roof.



Our own NumPy network on the scikit-learn digits: learning curves, centred against shifted digits, and the eight learned 3×3 filters of the first layer.

Why the same filter everywhere is the right choice: move the input, the map moves



The five is placed at two positions on a canvas; the vertical-edge map slides with it and is otherwise *identical* – shifting the second map back and subtracting gives exactly 0.0 (top). A filter that responds to an edge responds to that edge wherever it is, because the same weights are used at every position. A dense layer has no such property: it would have to learn the five at every position separately. (The bottom row shows that a stride of 2 keeps the property only for even shifts – one reason to stride with care.)

Pause and think 3: reading a feature map

Question

Why does the vertical-edge filter, whose rows read $(-1, 0, 1)$, light up on the *vertical* strokes of the five and of the house? What does a ReLU after the layer do to such a map? And what happens to the roof filter if the house in the picture is twice as large?

Pause and think 3: reading a feature map

Question

Why does the vertical-edge filter, whose rows read $(-1, 0, 1)$, light up on the *vertical* strokes of the five and of the house? What does a ReLU after the layer do to such a map? And what happens to the roof filter if the house in the picture is twice as large?

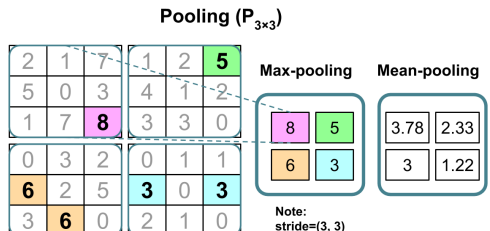
Answer

The filter is a difference between the pixel to the right and the pixel to the left – a derivative *along a row*. That derivative is large where the intensity changes as we move sideways, which is at a vertical stroke (the ink begins or ends), and zero on a horizontal stroke, where every row is constant. The sign says whether the ink begins (red) or ends (blue); a ReLU keeps only one of the two, so a trained network typically learns a filter *and* its negative. The roof filter is 10×16 pixels and matches a roof of that size: a roof twice as large gives a much weaker response. Convolution handles *position* for free; size and rotation it does not – a network learns those from examples at several scales, or the data are augmented.

Pooling

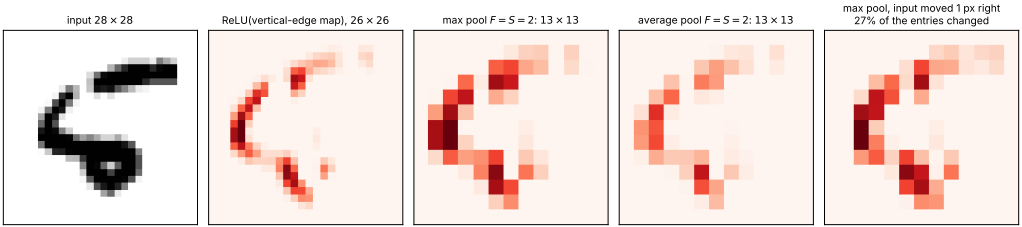
Besides convolutions, **pooling** is the other building block. A pooling layer reduces the size of the feature maps by summarising small regions: slide a window over the input and replace its content by one number – the **maximum** (max pooling, by far the most common) or the **average**.

- Works like a convolution, with the dot product replaced by a fixed function; *no parameters* to train.
- Usually non-overlapping windows: 2×2 with stride 2 halves width and height and keeps one of four numbers.
- Each channel is pooled separately: the depth does not change.
- Gives *invariance to small translations*: if the largest value moves within its window, the output does not change.



Max and mean pooling with 3×3 windows and stride 3 (Raschka et al.).

Pooling on our feature map (`week44_pooling.pdf`)



The vertical-edge map of the five after a ReLU (26×26), max-pooled and average-pooled with 2×2 windows and stride 2 (13×13), and the max-pooled map when the digit is moved one pixel to the right: 73% of the pooled values are unchanged. Max pooling keeps the strongest response in each window; average pooling blurs. After two conv–pool blocks a 28×28 digit has become a stack of 7×7 maps, which is flattened and handed to the dense layers.

Pause and think 4: follow the sizes through the MNIST network

Question

Input $28 \times 28 \times 1$. Conv with $K = 32$ filters, $F = 3$, $S = 1$, $P = 1$; max pool 2×2; conv with $K = 64$, $F = 3$, $S = 1$, $P = 1$; max pool 2×2; flatten; dense to 1024; dense to 10. Give the size of every volume and the number of parameters of every layer.

Pause and think 4: follow the sizes through the MNIST network

Question

Input $28 \times 28 \times 1$. Conv with $K = 32$ filters, $F = 3$, $S = 1$, $P = 1$; max pool 2×2; conv with $K = 64$, $F = 3$, $S = 1$, $P = 1$; max pool 2×2; flatten; dense to 1024; dense to 10. Give the size of every volume and the number of parameters of every layer.

Answer

$28 \times 28 \times 1 \to 28 \times 28 \times 32$ (320 parameters) $\to 14 \times 14 \times 32$ (pool, 0) $\to 14 \times 14 \times 64$ $(64 \times (3 \cdot 3 \cdot 32 + 1) = 18\,496) \to 7 \times 7 \times 64$ (pool, 0) $\to 3136$ (flatten) $\to 1024$ $(3136 \times 1024 + 1024 = 3\,212\,288) \to 10$ (10 250). Total 3 241 354, of which the two convolutional layers hold 18 816, less than one per cent. The dense layer after the flattening holds almost everything – keep this in mind when we look at the results.

Building a CNN with TensorFlow/Keras or PyTorch: setting it up

- The data are no longer a 2D feature matrix. Each input is a stack of 2D arrays, one per colour channel, and the whole data set is a 4D array (a *tensor*):

$$(n_{\text{inputs}},\ n_{\text{pixels, width}},\ n_{\text{pixels, height}},\ \text{depth})$$

  in Keras; PyTorch puts the depth (channels) *first*, $(n_{\text{inputs}}, \text{depth}, \text{height}, \text{width})$.
- MNIST: 28×28 grey images, depth 1, so a tensor of shape $(60\,000, 28, 28, 1)$ – the same 784 pixels per image as in Project 2, only not flattened.
- Images have strong *local correlations*: a small part of the image varies little from its neighbours. So instead of connecting every pixel to every neuron, each neuron sees a small region – its *receptive field* – in all channels.
- Convolution (with a ReLU) and pooling are stacked in pairs. By systematically reducing the size of the volume, the network builds representations of small parts of the image and assembles them into larger ones. The last pooled volume is flattened and fed to ordinary dense layers with a softmax output.

Training: nothing new

- A convolutional layer is a linear map (the matrix W') followed by an activation; pooling is a fixed function. The network is a composition of differentiable functions, so **backpropagation works unchanged**: the gradient with respect to the filter weights is itself a convolution (of the input with the error), and the gradient through a max pool goes to the element that won the maximum.
- The optimiser is the one of weeks 38–41: SGD or Adam on minibatches, with the softmax cross entropy of Chapter 5.
- The libraries do all of this by automatic differentiation (`loss.backward()` in PyTorch, `model.fit` in Keras).

In the notebook, for those who want to see it

`week44.ipynb` contains a complete NumPy implementation – the convolution as one matrix product (`im2col`), the backward pass, max pooling and Adam – with its gradients checked against finite differences and the same network run on the digits and on MNIST. It is there to show that there is nothing in the library we could not write ourselves; it is *not* needed for Project 2.

A complete TensorFlow/Keras program for MNIST

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
keras.utils.set_random_seed(1)

Xtr, ytr, Xte, yte = mnist_arrays() # (N,1,28,28) in [0,1], from mnist.npz
std = lambda X: ((X - 0.1307) / 0.3081).astype("float32").transpose(0, 2, 3, 1)
Xtr, Xte = std(Xtr), std(Xte) # (N, 28, 28, 1): channels LAST

model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, padding="same", activation="relu"), # 28x28x32, 320
           param.
    layers.MaxPooling2D((2, 2)), # 14x14x32
    layers.Conv2D(64, 3, padding="same", activation="relu"), # 14x14x64, 18496
    layers.MaxPooling2D((2, 2)), # 7x7x64
    layers.Flatten(), # 3136
    layers.Dense(1024, activation="relu"), # 3212288
    layers.Dropout(0.5),
    layers.Dense(10)]) # 10250, logits
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.summary() # prints the parameter counts

for epoch in range(5):
    model.fit(Xtr, ytr, epochs=1, batch_size=64, verbose=0)
    print(epoch + 1, model.evaluate(Xte, yte, verbose=0)[1])
```

Reading the program

- `Conv2D(K, F, padding="same")`: the cross-correlation of Part 2 with K filters of size $F \times F$ and the padding that keeps the size; `MaxPooling2D((2, 2))`: the pooling.
- The comments give the volume after each layer and the parameter counts of Pause and think 4; `model.summary()` prints them.
- `from_logits=True`: the network ends with scores and the loss applies the softmax itself.
- `Dropout(0.5)`: half of the 1024 units switched off at random in each training step – a regulariser for the layer that holds 99% of the parameters; off at test time.

The same network in PyTorch

```
import torch, torch.nn as nn
torch.manual_seed(1)
Xtr, ytr, Xte, yte = mnist_arrays() # (N,1,28,28): channels FIRST
std = lambda X: torch.tensor(((X - 0.1307) / 0.3081).astype("float32"))
Xtr, Xte, Ytr, Yte = std(Xtr), std(Xte), torch.tensor(ytr), torch.tensor(yte)

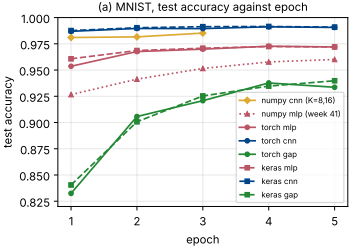
net = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # (N,32,14,14)
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # (N,64,7,7)
    nn.Flatten(), nn.Linear(64 * 7 * 7, 1024), nn.ReLU(),
    nn.Dropout(0.5), nn.Linear(1024, 10)) # logits
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
lossf = nn.CrossEntropyLoss() # applies the softmax itself

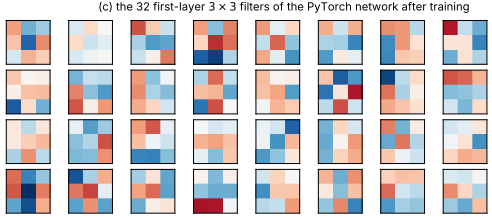
for epoch in range(5):
    net.train() # dropout on
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr) - 64 + 1, 64):
        idx = perm[i:i + 64]
        opt.zero_grad(set_to_none=True)
        loss = lossf(net(Xtr[idx]), Ytr[idx])
        loss.backward() # backpropagation, by autograd
        opt.step()
    net.eval() # dropout off
    with torch.no_grad():
        pred = torch.cat([net(Xte[i:i + 1000]).argmax(1) for i in range(0, 10000,
                1000)])
    print(epoch + 1, (pred == Yte).double().mean().item())
```

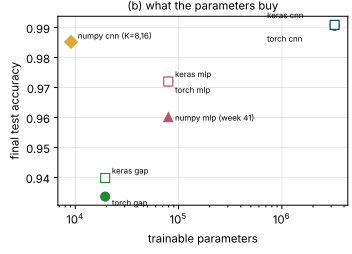
Reading the program

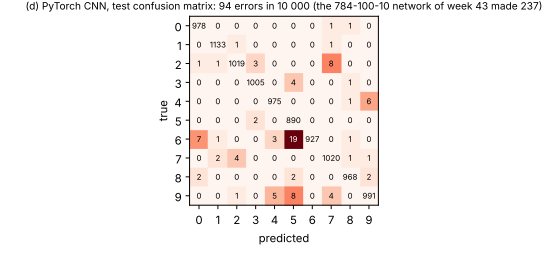
- `nn.Conv2d(C_in, K, F, padding=P)`: input channels, filters, filter size, padding; layout (N, C, H, W), channels first – the one structural difference from Keras.
- PyTorch shows the loop that `model.fit` hides: a minibatch, the loss, `backward()`, a step of Adam.
- `nn.CrossEntropyLoss` expects logits; ending with a softmax applies it twice (a common, silent bug).
- `train()`/`eval()` switch dropout on and off; forget `eval()` and the test accuracy is measured with half the units missing.
- The dense baseline of Project 2 is one line: `nn.Sequential(nn.Flatten(), nn.Linear(784, 100), nn.ReLU(), nn.Linear(100, 10))`, same loop.

Results on MNIST (week44_mnist.pdf)









Reading the results

Five epochs, Adam 10^{-3}, batches of 64, seed 1; seconds per epoch on one CPU

code	model	param.	test acc.	s/ep.
NumPy	dense 784–100–10, SGD (week 41)	79 510	0.9601	1
PyTorch	dense 784–100–10, Adam	79 510	0.9720	1
Keras	dense 784–100–10, Adam	79 510	0.9720	3
PyTorch	CNN of the listings	3 241 354	0.9906	47
Keras	CNN of the listings	3 241 354	0.9909	37
PyTorch	CNN, global average pooling	19 466	0.9336	19
Keras	CNN, global average pooling	19 466	0.9398	18
NumPy	CNN $K = 8, 16$ (3 epochs)	9 098	0.9852	71

- **CNN against the Project 2 network:** same data, same optimiser, 94 errors instead of 280 on the 10 000 test images (week 43's dense network: 237); the week-43 error bar $\sqrt{p(1-p)/n} \approx 0.001$ says the gap is real.
- **PyTorch and Keras agree** to the noise of a stochastic optimiser (0.9906 against 0.9909): different initialisations, shuffles and dropout masks, the same model.
- **Where the parameters are:** replace the 3.2-million dense layer by the average of each 7×7 map (global average pooling, 19 466 parameters) and we lose five points in five epochs – 99% of the network does none of the convolutional work, but it is not pure waste either.
- **Our NumPy CNN** with 9 098 parameters beats the 79 510-parameter dense network: the *architecture* is what the digits reward, not the count. Forty times slower per epoch – the library is the tool.

Does it always help? The experiment of Section 10.10

The scikit-learn digits (8×8 pixels, 1797 images): our NumPy CNN against a dense network with the same number of parameters, five seeds.

data	model	parameters	test accuracy
centred 8×8	CNN	1898	0.966
centred 8×8	dense	1885	0.967
random offset in 12×12	CNN	2698	0.877
random offset in 12×12	dense	6830	0.707

- On the digits as supplied – small, centred, scaled – the CNN is *not* better. There is no position to be invariant to, and a dense layer can afford one weight per pixel.
- Put the same digits at a random place in a slightly larger frame and the dense network loses 26 points, the CNN 9: the CNN learns each stroke once and finds it wherever it is; the dense network must learn every digit at every position.
- So: convolution is the right tool when the data have the structure it assumes – patterns that can appear anywhere on a grid. Images do; a table of unrelated measurements does not.

Try at home: colour images, the CIFAR-10 data set (from the 2025 notes)

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.
       load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0 #
       (50000,32,32,3)

model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
                  ,
              metrics=['accuracy'])
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

- 60 000 colour images of $32 \times 32 \times 3$ in ten classes (airplane, automobile, bird, cat, ...), 50 000 for training.
- The first `Conv2D` has $3 \times 3 \times 3 + 1 = 28$ parameters per filter, 896 in all – the answer of Pause and think 2. With `padding` left at its default ("valid"), the sizes go $32 \to 30 \to 15 \to 13 \to 6 \to 4$.
- The program downloads the data (170 MB) and reaches about 70% test accuracy in ten epochs – a much harder problem than MNIST, and where the dense network of Project 2 is hopeless.

Fourier filters: the convolution theorem

The Fourier transform

$$\hat{f}(\omega) = \int_{-\infty}^{\infty} f(t)\, e^{-i\omega t}\, dt, \qquad f(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} \hat{f}(\omega)\, e^{i\omega t}\, d\omega,$$

and for N samples the discrete transform (what `np.fft.fft` computes),

$$X_k = \sum_{n=0}^{N-1} x_n\, e^{-2\pi i k n / N}, \qquad x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k\, e^{2\pi i k n / N}.$$

The theorem

For a convolution $(f * h)(t) = \int f(\tau)\, h(t-\tau)\, d\tau$, substitute $s = t - \tau$ and the double integral factorises:

$$\boxed{\widehat{f * h}(\omega) = \hat{f}(\omega)\, \hat{h}(\omega)}$$

Convolution in t is *multiplication* in ω. For sampled data the same holds with the DFT and the *circular* convolution $(h \circledast x)_n = \sum_m h_m\, x_{(n-m) \bmod N}$.

Why this matters for filters

A convolution filter $y = h * f$ therefore multiplies every frequency component by one number,

$$\hat{y}(\omega) = H(\omega)\, \hat{f}(\omega), \qquad H = \hat{h},$$

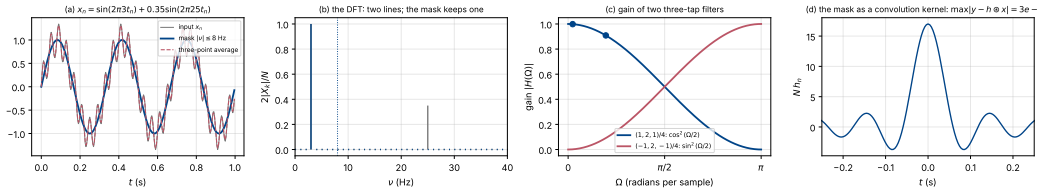
the *frequency response*: $|H(\omega)|$ is the gain of the mode $e^{i\omega t}$, $\arg H$ its phase shift. Fourier modes are the eigenfunctions of every convolution: $(h * e^{i\omega \cdot})(t) = H(\omega)\, e^{i\omega t}$.

Two ways to filter, one operation

- in t: convolve with a kernel h (what a CNN layer does);
- in ω: transform, multiply by H, transform back.

The *same* linear map: the second is how one designs or understands a filter, the first is how a network applies it.

Example: a two-tone signal, a frequency mask, a three-point average (`week44_fourier1d.pdf`)



(a) $x_n = \sin(2\pi\, 3t_n) + 0.35\sin(2\pi\, 25t_n)$, $N = 256$ samples in one second. (b) Its DFT has two lines, at 3 and 25 Hz, with the two amplitudes. A *mask* $H_k = 1$ for $|\nu_k| \le 8$ Hz, 0 otherwise, multiplied onto X_k and transformed back, returns $\sin(2\pi\, 3t_n)$ to 2×10^{-15}: the 25 Hz tone is gone. (c) The three-point average $y_n = \frac{1}{4}x_{n-1} + \frac{1}{2}x_n + \frac{1}{4}x_{n+1}$ is a convolution with $h = (1, 2, 1)/4$, whose response is $H(\Omega) = \cos^2(\Omega/2)$, $\Omega = 2\pi\nu/N$: gain 0.999 at 3 Hz, 0.909 at 25 Hz – a gentle low pass; $(-1, 2, -1)/4$ has $\sin^2(\Omega/2)$, the complementary high pass. (d) The mask is *also* a convolution: its kernel $h = \mathrm{IDFT}(H)$ is a sampled $\sin(\omega_c t)/(\pi t)$ with long oscillating tails, and $h \circledast x$ reproduces the masked result to 3×10^{-16}. Sharp in frequency means wide and oscillatory in time – the origin of "ringing" near edges.

Linear against circular convolution, and the FFT as a fast convolution

The DFT treats the record as periodic, so the product $H_k X_k$ is a *circular* convolution. For $x = (1, 2, 3)$ and $h = (1, 1)$ the linear convolution is $x * h = (1, 3, 5, 3)$, but

FFT length	result	
$N = 3$	$(4, 3, 5)$	the last 3 wrapped into the first bin
$N = 4$	$(1, 3, 5, 3)$	the full linear convolution

Pad both sequences with zeros to $N \geq L_x + L_h - 1$ and the FFT reproduces the linear convolution exactly:

```
x, h = np.array([1., 2., 3.]), np.array([1., 1.])
N = len(x) + len(h) - 1
y = np.fft.ifft(np.fft.fft(x, N) * np.fft.fft(h, N)).real # [1. 3. 5.
        3.]
```

Cost

Direct convolution of N samples with K taps costs $O(NK)$; three FFTs cost $O(N \log N)$. For the 3×3 filters of a CNN the direct product wins (and the libraries use it); for long kernels – the sinc of the previous frame, large blurs, audio filters – the FFT wins by orders of magnitude.

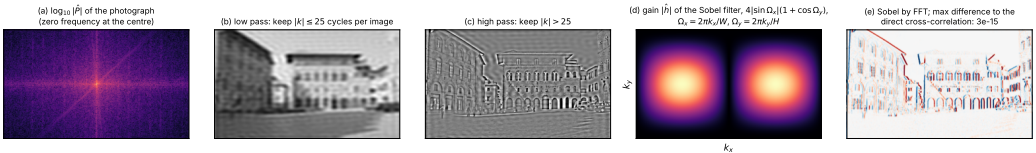
Padding is the boundary model

Circular: the DFT diagonalises the filter directly. Zero padding (the P of a CNN layer): pad to $N \geq L_x + L_h - 1$, then crop. Reflection: extend the data first.

Cross-correlation in Fourier language

A CNN layer computes $z_n = \sum_r w_r x_{n+r}$, the convolution with the *reversed* kernel $h_r = w_{-r}$, so its response is $A_k = \overline{W_k}$ for real weights: the same gain, the opposite phase shift. Symmetric kernels ($(1, 2, 1)$, the blur) do not notice.

The photograph in Fourier space (`week44_fourier2d.pdf`)



(a) $\log_{10}|\hat{P}(k_x, k_y)|$ of the 211×360 photograph: most of the energy sits at low frequencies, with the bright horizontal and vertical lines of the many vertical and horizontal edges. (b) Keep only $\sqrt{k_x^2 + k_y^2} \le 25$ cycles per image – 2.6% of the coefficients – and transform back: a blur, the low-pass filter of Part 3 done by multiplication. (c) Keep the rest: the edges, a high-pass filter. (d) The frequency response of the Sobel filter, $|\hat{h}| = 4|\sin\Omega_x|\,(1+\cos\Omega_y)$: zero on the axis $\Omega_x = 0$ (a horizontal edge is invisible to it), largest at $|\Omega_x| = \pi/2$, and attenuated at high Ω_y by the $(1, 2, 1)$ smoothing. (e) The Sobel map computed as $\mathrm{IDFT}(\hat{h}\,\hat{P})$ agrees with the direct cross-correlation to 3×10^{-15}: the two routes are one operation.

A CNN layer in Fourier language

The linear stage is channel mixing, frequency by frequency

With circular boundaries and stride 1, the layer $z_a[n] = b_a + \sum_b \sum_r w_{ab}[r]\, x_b[n+r]$ becomes, after a DFT,

$$Z[k] = A[k]\, X[k], \qquad A[k] \in \mathbb{C}^{C_{\text{out}} \times C_{\text{in}}},$$

one small matrix per frequency, with entries $A_{ab}[k] = \sum_r w_{ab}[r]\, e^{2\pi i k r / N}$. Different frequencies never mix in the linear stage.

What a short kernel can and cannot do

A 3×3 kernel has a response that is a trigonometric polynomial of degree one in each direction – the smooth surfaces of panel (d). Its frequency bins cannot be set independently: a sharp cutoff like the mask of the 1D example needs a long kernel, or a frequency-space layer.

Where the frequency picture stops

- **The ReLU creates new frequencies**: $\max(0, \sin t) = \frac{1}{2}\sin t + \frac{1}{2}|\sin t|$ has a constant term and all even harmonics. A full CNN has no single response H; the linear stage of each layer does.
- **Stride and pooling change the sampling grid**: keeping every second sample folds frequencies above the new Nyquist limit back down – $\cos(0.2\pi n)$ and $\cos(0.8\pi n)$ become identical. A low pass before downsampling (a blur before the pool) removes the aliases; a learned strided filter does not do this by itself.

Training in Fourier words

Gradient descent on $w[r]$ moves the response $A(\Omega) = \sum_r w[r] e^{i\Omega r}$ at all frequencies at once. What it converges to depends on the task: the edge detectors of Part 3 are directional high-pass filters, learned because edges separate digits.

How Fourier filters are used, 1: signal processing and differential equations

- **Denoising and band selection.** A mask or a smooth response H removes what lies outside the band of interest: the 25 Hz tone of the example, the stone texture of the photograph, the mains hum in a measurement. The choice of H is the model of what is signal and what is noise; the transform itself decides nothing.
- **Fast convolution.** A long kernel – a wide blur, a sinc, an audio reverberation – is applied with three FFTs in $O(N \log N)$ instead of $O(NK)$.
- **Solving differential equations.** The Poisson problem of week 43, $-u'' = f$ on $(0,1)$ with $u(0) = u(1) = 0$: expand in the sine modes that satisfy the boundary conditions, $f = \sum_n f_n \sin(n\pi x)$, and since $-\frac{d^2}{dx^2}\sin(n\pi x) = (n\pi)^2 \sin(n\pi x)$,

$$u(x) = \sum_{n=1}^{\infty} \frac{f_n}{(n\pi)^2} \sin(n\pi x) :$$

  the inverse operator is a *filter* that damps mode n by $1/(n\pi)^2$. (Fixed boundaries break translation invariance, so this is a sine series, not a convolution on the whole line.)
- **Reading a filter.** The response H of a kernel says what it does – low pass, high pass, oriented – more directly than its weights: $(1,2,1)/4$ is $\cos^2(\Omega/2)$, Sobel is $4|\sin\Omega_x|(1+\cos\Omega_y)$.

How Fourier filters are used, 2: inside convolutional networks

- **Anti-aliasing.** A blur before every stride-2 pool, so that a shifted input gives a shifted output instead of an aliased one (Zhang, ICML 2019).
- **Spectral pooling.** Reduce the resolution by truncating the DFT of a feature map instead of taking maxima: the low frequencies are kept exactly (Rippel, Snoek and Adams, NeurIPS 2015).
- **Learning the filters in frequency space.** $z = \mathrm{IDFT}\big[R_\theta[k]\, X[k]\big]$ with one learned matrix per retained frequency – a *global* kernel with few parameters. This is the layer of the Fourier neural operator, which learns the solution operator of a PDE from examples (Li et al. 2021).
- **Fourier-weighted losses** for reconstruction tasks. By Parseval, $\frac{1}{N}\sum_n |e_n|^2 = \frac{1}{N^2}\sum_k |E_k|^2$ for the error $e = y - t$; weights q_k on the spectrum decide which frequencies the training must get right.

What to remember

A Fourier transform is a change of representation and loses nothing; a filter is a choice of what to keep. Convolution with h and multiplication by $H = \hat{h}$ are the same operation. A CNN learns H one short kernel at a time, with non-linearities and pooling in between that the frequency picture does not cover. The derivations are in Section 10.13 of the lecture notes.

Pause and think 5: two sequences through two filters

Question

Send the alternating sequence $x_n = (-1)^n$ and the constant sequence $x_n = 1$ through the three-point average $(1, 2, 1)/4$ and through $(-1, 2, -1)/4$. What comes out, and why does the Sobel response vanish on the line $\Omega_x = 0$?

Pause and think 5: two sequences through two filters

Question

Send the alternating sequence $x_n = (-1)^n$ and the constant sequence $x_n = 1$ through the three-point average $(1, 2, 1)/4$ and through $(-1, 2, -1)/4$. What comes out, and why does the Sobel response vanish on the line $\Omega_x = 0$?

Answer

The alternating sequence is the mode $\Omega = \pi$: the average has $\cos^2(\pi/2) = 0$, so it is removed entirely ($\frac{1}{4} - \frac{1}{2} + \frac{1}{4} = 0$ at every n); $(-1, 2, -1)/4$ has $\sin^2(\pi/2) = 1$ and passes it unchanged. The constant sequence is $\Omega = 0$: the average passes it ($\cos^2 0 = 1$) and the high pass kills it ($-\frac{1}{4} + \frac{1}{2} - \frac{1}{4} = 0$). The two gains add to one – the filters split the signal into its smooth and its varying part. The Sobel filter's columns are $(-1, 0, 1)$: a central difference along the row, whose response is $2i \sin \Omega_x$ – zero at $\Omega_x = 0$, which is every pattern that does not change along a row. A horizontal edge is such a pattern, so it is invisible to W_{vert} and needs W_{hor} – exactly what the house showed.

Where this is going: tomorrow, the exercises, Project 2

Tomorrow: cross-validation for your Project 2 network

Twenty minutes of theory – what the k-fold estimate is, its error bar, the one-standard-error rule, why the scaler goes inside the fold – then the six questions of exercise 1 and your own network inside `scikit-learn`'s `KFold`, `cross_validate`, `GridSearchCV` and `validation_curve` (exercise 2).

Exercises week 44 (deadline Friday October 30)

1: six questions on cross-validation. 2: your Project 2 regression network cross-validated by `scikit-learn`. 3 (optional): the MNIST classifier. 4 (optional): the loop written by hand. Declare your use of LLMs.

Project 2 (deadline November 6)

Asks for a dense network and logistic regression, not a CNN. A group that wants to add today's CNN as a comparison is welcome to – cross-validated the same way, and compared honestly.

Next Monday, week 45

Chapter 11, recurrent neural networks – sequences, the unrolled graph, backpropagation through time – and time for Project 2 questions.

Reading

Chapter 10, Sections 10.1–10.7 and 10.10–10.13 (10.13: Fourier filters); Goodfellow et al. Chapter 9; Raschka et al. Chapters 14–15. For tomorrow: Sections 2.12, 3.14 and 8.12.

Summary of week 44

- A CNN is a neural network that *assumes its input is an image*: neurons see a small neighbourhood (locality) and the same weights are used everywhere (parameter sharing). The parameter count no longer depends on the image size.
- Convolution: $y(i) = \sum_k w(k)\, x(i-k)$ – polynomial multiplication is the first example, a Toeplitz matrix the second way to write it. Padding supplies the missing indices, the stride decides how far the filter moves, and the libraries compute the cross-correlation $\sum_k w(k)\, x(i+k)$ under the name convolution.
- In two dimensions: slide an $F \times F$ filter over the image, dot product at every position; output size $(W_1 - F + 2P)/S + 1$, depth K, parameters $(F \times F \times D_1)K + K$.
- Filters detect things: edges on a digit and a house, a roof, a window in a photograph. A CNN *learns* its filters, and learns edge detectors first. Pooling discards position in small steps.
- PyTorch and Keras: a few lines for the network, the familiar loop for training; 99% on MNIST against 97% for the Project 2 network, with the same optimiser. Every number has hyperparameters behind it that nobody chose – tomorrow is about choosing them.
- Fourier: convolution with h is multiplication by $H = \hat{h}$; a filter is a choice of which frequencies to keep; a short CNN kernel is a smooth response, learned one weight at a time, with non-linearities and pooling in between that Fourier does not see.